

Real time research project report
on
Handwritten Alphabet Recognition System
submitted in partial fulfillment of the requirements for the award of degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING
(AI&ML)

by

24WH1A6604 Ms. B. RASHMITHA
24WH1A6616 Ms. S. NIKITHA
24WH1A6626 Ms. E. NIHARIKA
24WH1A6642 Ms. K. NITHYA

under the esteemed guidance of
MS. A. NAGA KALYANI
Assistant Professor, CSE(AI&ML)



Department of Computer Science & Engineering
(Artificial Intelligence & Machine Learning)

BVRIT HYDERABAD COLLEGE OF ENGINEERING FOR WOMEN
(NAAC Accredited-A Grade | NBA Accredited B.Tech. (EEE, ECE, & CSE)
(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Bachupally, Hyderabad – 500090

2025 - 2026

BVRIT HYDERABAD COLLEGE OF ENGINEERING FOR WOMEN

(NAAC Accredited-A Grade | NBA Accredited B.Tech. (EEE, ECE, & CSE)

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Bachupally, Hyderabad– 500090

Computer Science & Engineering (Artificial Intelligence & Machine Learning)



CERTIFICATE

This is to certify that the Real time research project entitled “**HANDWRITTEN ALPHABET RECOGNITION SYSTEM**” is a bonafide work carried out **by Ms. B. RASHMITHA (24WH1A6604), Ms. S. NIKITHA (24WH1A6616), Ms. E. NIHARIKA (24WH1A6626), Ms. K. NITHYA (24WH1A6642)**, in partial fulfillment for the award of B.Tech degree in **Computer Science & Engineering (AI&ML), BVRIT HYDERABAD College of Engineering for Women, Bachupally, Hyderabad**, affiliated to Jawaharlal Nehru Technological University, Hyderabad under my guidance and supervision. The results embodied in the project work have not been submitted to any other.

Internal Guide

Ms. A. Naga Kalyani
Assistant Professor,
Department of CSE(AI&ML)

Head of the Department

Dr. B. Lakshmi Praveena
Professor and HoD,
Department of CSE(AI&ML)

External Examiner

DECLARATION

We hereby declare that the work presented in this project entitled “**Handwritten Alphabet Recognition System**” submitted towards completion of Real Time Research Project in II year of B.Tech, II semester, CSE(AI&ML) at ‘**BVRIT HYDERABAD College of Engineering for Women**’, Hyderabad is an authentic record of our original work carried out under the guidance of **Ms. A. Naga Kalyani, Assistant Professor, Department of CSE(AI&ML).**

Ms. B. RASHMITHA
(24WH1A6604)

Ms. S. NIKITHA
(24WH1A6616)

Ms. E. NIHARIKA
(24WH1A6626)

Ms. K. NITHYA
(24WH1A6642)

ACKNOWLEDGEMENT

We would like to express our sincere thanks to **Dr. K. V. N. Sunitha**, Principal, BVRIT HYDERABAD College of Engineering for Women, for her support by providing the working facilities in the college.

Our sincere thanks and gratitude to **Dr. B. Lakshmi Praveena, Head of the Department, Department of CSE(AI&ML)**, BVRIT HYDERABAD College of Engineering for Women, for all timely support and valuable suggestions during the period of our project. We are extremely thankful to our Internal Guide, **Ms. A. Naga Kalyani, Assistant Professor, Department of CSE(AI&ML)**, BVRIT HYDERABAD College of Engineering for Women, for her constant guidance and encouragement throughout the project.

We would like to thank our **Real Time Project Coordinators, Ms. A. Naga Kalyani, Assistant Professor, Department of CSE(AI&ML) and Ms. K. Lalitha, Assistant Professor, Department of CSE(AI&ML)** and all the Staff of Department of CSE(AI&ML) department who helped us directly or indirectly throughout the project.

Finally, last but not least, we wish to acknowledge our Parents and Friends for giving moral strength and constant encouragement

B. RASHMITHA (24WH1A6604)

S. NIKITHA(24WH1A6616)

E. NIHARIKA(24WH1A6626)

K. NITHYA(24WH1A6642)

ABSTRACT

Handwritten alphabet recognition is a significant problem in the field of artificial intelligence and machine learning due to the high variability in individual writing styles and input conditions. Variations in stroke thickness, orientation, spacing, and drawing speed make accurate recognition a challenging pattern recognition task. Developing systems that can reliably interpret handwritten alphabets in real time remains essential for advancing human-computer interaction.

This project addresses the problem of recognizing handwritten English alphabets drawn on a digital interface. The objective is to enable accurate identification of user-drawn characters despite stylistic differences and irregularities in input. The system focuses on real-time recognition within a browser-based environment, emphasizing efficiency and responsiveness. Handwritten character recognition has wide-ranging applications in educational tools, digital note taking systems, automated form processing, document digitization, assistive technologies, and interactive learning platforms. By enabling machines to interpret handwritten input effectively, such systems contribute to improving accessibility, productivity, and seamless interaction between users and intelligent digital systems.

LIST OF CONTENTS

S NO	CONTENT	PAGE NO
1.	Introduction	1
	Problem Statement	2
	Objectives	3
	Background	4
2.	Literature Survey	5
3.	System Requirements	6-7
	Functional Requirements	6
	Non Functional Requirements	6
	Software Requirements	7
	Hardware Requirements	7
4.	Proposed System Analysis	8-9
	a. Existing Systems	8
	b. Disadvantages	8
	c. Proposed System	8
	d. Advantages	9
	e. Proposed System Architecture	9
5.	Implementation	10-12
	Training Code	10
	Testing Code	11
6.	System Architecture	13
7.	Result Analysis	14-15
8.	Conclusion	16
9.	References	17

INTRODUCTION

Handwritten alphabet recognition (HAR) is an important application of artificial intelligence, particularly in the fields of deep learning and computer vision. It focuses on enabling machines to interpret and convert handwritten English alphabets into a digital format. This process is challenging due to the wide variation in individual handwriting styles, including differences in stroke thickness, spacing, orientation, and writing speed.

With recent advancements in deep learning, Convolutional Neural Networks (CNNs) have become highly effective for image-based recognition tasks. CNNs can automatically extract important features such as edges, curves, and patterns from input images, eliminating the need for manual feature engineering and improving overall accuracy.

In this project, a CNN-based model is developed to recognize handwritten English alphabets (A–Z) using the EMNIST dataset. Image preprocessing techniques such as normalization, resizing, and noise removal are applied to enhance the quality of the input data. Additionally, data augmentation is used to improve the model's ability to handle variations in handwriting styles.

The system also incorporates real-time input processing using OpenCV, allowing users to draw alphabets on a digital interface and receive immediate predictions. This demonstrates the practical application of deep learning in building accurate, efficient, and interactive handwritten alphabet recognition systems.

PROBLEM STATEMENT

Handwritten alphabet recognition remains a challenging task due to the high variability in individual writing styles and input conditions. Differences in stroke thickness, size, orientation, spacing, and writing speed make it difficult for traditional systems to accurately interpret handwritten English alphabets. Additionally, variations caused by noise, low-quality input images, and inconsistent drawing patterns further reduce recognition accuracy.

Existing methods often struggle to generalize across diverse handwriting styles and may not perform well in real-time applications. There is a need for an efficient and accurate system that can reliably recognize handwritten English alphabets (A–Z) from user input.

This project aims to develop a robust handwritten alphabet recognition system using Convolutional Neural Networks (CNNs) trained on the EMNIST dataset. The system should effectively preprocess input images, handle variations in handwriting through data augmentation, and provide accurate predictions in real time using a user-friendly interface.

OBJECTIVES

1. **Develop an Accurate Alphabet Recognition Model:**
Design and implement a Convolutional Neural Network (CNN) model to accurately recognize handwritten English alphabets (A–Z).
2. **Utilize Standard Dataset for Training:**
Use the EMNIST dataset to train and validate the model, ensuring reliable performance across diverse handwriting styles.
3. **Apply Image Preprocessing Techniques:**
Implement preprocessing methods such as normalization, resizing, and noise removal to improve input image quality and model accuracy.
4. **Enhance Model Generalization using Data Augmentation:**
Apply data augmentation techniques like rotation, scaling, and shifting to handle variations in handwriting styles.
5. **Enable Real-Time Alphabet Recognition:**
Develop a system that captures user input in real time and predicts the corresponding alphabet instantly.
6. **Integrate OpenCV for Input Processing:**
Use OpenCV to process user-drawn alphabets from a digital interface for real-time prediction.
7. **Improve Model Performance and Efficiency:**
Optimize the CNN model to achieve high accuracy while maintaining low computational cost and fast response time.
8. **Ensure Robustness to Variations and Noise:**
Build a system that performs reliably under different input conditions, including noisy or distorted handwritten inputs.
9. **Develop a User-Friendly Interface:**
Create an interactive interface that allows users to easily draw alphabets and view predictions.
10. **Design a Scalable and Extendable System:**
Develop the system in a modular way to allow future enhancements, such as extending recognition to digits, words, or multiple languages.

BACKGROUND

Handwritten alphabet recognition (HAR) is a fundamental problem in the field of pattern recognition and has been widely studied as part of artificial intelligence and machine learning. The goal of HAR is to enable computers to automatically identify and interpret handwritten alphabets from images or digital inputs. This capability is essential for converting handwritten information into machine-readable formats, reducing manual effort and improving efficiency.

Earlier approaches to handwritten recognition relied on rule-based systems and template matching techniques, where predefined patterns were compared with input images. These methods were simple but lacked flexibility and performed poorly when faced with variations in handwriting styles. Later, traditional machine learning algorithms such as Support Vector Machines (SVM) and k-Nearest Neighbors (k-NN) were introduced, which improved performance by learning from data. However, these methods depended heavily on manual feature extraction, making them less effective for complex and diverse inputs.

The emergence of deep learning, particularly Convolutional Neural Networks (CNNs), has significantly advanced handwritten recognition systems. CNNs are capable of automatically extracting hierarchical features from raw image data, allowing them to capture complex patterns such as curves, edges, and shapes in handwritten alphabets. This has led to substantial improvements in accuracy and robustness compared to traditional methods.

Datasets like EMNIST, which extend the well-known MNIST dataset, provide large collections of labeled handwritten alphabets and are widely used for training and evaluating recognition models. In addition, preprocessing techniques such as normalization, resizing, and noise removal help standardize input data, while data augmentation improves the model's ability to generalize across different handwriting styles.

Today, handwritten alphabet recognition systems are increasingly integrated with real-time applications using tools like OpenCV, enabling interactive systems where users can draw alphabets and receive instant predictions. These advancements have made HAR a practical and widely used solution in various digital applications.

LITERATURE SURVEY

S.No.	Author(s) & Year	Method Used	Dataset	Key Contribution	Limitations
1.	LeCun et al., 1998	Convolutional Neural Networks (CNN)	MNIST	Introduced CNN for handwritten recognition and achieved high accuracy in classification.	Limited to digit recognition (0–9) only.
2.	Cohen et al., 2017	Deep Learning	EMNIST	Extended MNIST dataset to include alphabets (A–Z) and made it suitable for alphabet recognition tasks.	Class imbalance in certain alphabet categories.
3.	Simard et al., 2003	Data Augmentation	MNIST	Improved recognition performance by applying distortions and data augmentation to training images.	Focused mainly on digits, not alphabets.
4.	Ciresan et al., 2012	Deep Neural Networks (DNN)	MNIST	Achieved near-human accuracy in handwritten recognition using deep neural network architectures.	High computational cost and training time.
5.	Zhang et al., 2018	Optimized CNN Model	EMNIST	Proposed an optimized CNN model that improved accuracy for handwritten alphabet recognition.	Requires large training data and careful parameter tuning.

SYSTEM REQUIREMENTS

Functional Requirements

- The system must accept handwritten alphabet input in the form of images or real-time drawing (via mouse, touchscreen, or stylus).
- The platform should preprocess input images (grayscale conversion, resizing, normalization, noise removal).
- The system must accurately recognize and classify handwritten alphabets (A–Z).
- The model should support both offline prediction (uploaded images) and real-time recognition.
- The system must display prediction results along with confidence scores.
- The platform should allow dataset loading, training, and testing of the model.
- User interface should be simple and interactive for easy input and visualization of results.
- The system must store trained models and allow reuse without retraining.

Non-Functional Requirements

- The system should achieve high accuracy (preferably above 90%) for reliable predictions.
- The application must provide fast response time for real-time recognition.
- The system should be scalable to support larger datasets and extended character sets.
- The platform must ensure data security and integrity during input processing.
- The system should be robust against noise and variations in handwriting styles.
- The application should be user-friendly and easy to operate.
- The system should be portable across different operating systems.

3.1 Software Requirements

Operating System

- Windows 10/11
- Linux (Ubuntu recommended for development)
- macOS

Primary Languages

- Python (main programming language for model development)
- Optional: JavaScript (for web-based UI if implemented)

Development Environment (IDE)

- Jupyter Notebook / Google Colab (for model training)
- Visual Studio Code / PyCharm

Libraries & Frameworks

- TensorFlow / Keras – for building deep learning models
- PyTorch (alternative to TensorFlow)
- NumPy – numerical computations
- Pandas – dataset handling
- Matplotlib / Seaborn – visualization
- OpenCV – image processing

Dataset

- EMNIST Dataset (Extended MNIST for alphabets)
- MNIST Dataset (for basic digit reference)
- Custom handwritten dataset (optional)

Version Control

- Git (for code management and collaboration)

3.2 Hardware Requirements

- Processor: Intel Core i5 or higher (multi-core recommended)
- RAM: Minimum 8 GB (16 GB recommended for training deep learning models)
- Storage: At least 256 GB SSD for faster processing and storage of datasets/models
- GPU (Optional): NVIDIA GPU with CUDA support for faster training
- Input Devices: Keyboard, mouse, touchscreen, or stylus for handwriting input
- Network Connectivity: Required for downloading datasets, libraries, and cloud-based training (e.g., Google Colab)

PROPOSED SYSTEM ANALYSIS

a) Existing Systems

- Traditional OCR systems based on **rule-based methods**
- Machine learning models like SVM, KNN
- Basic neural networks without deep learning
- Systems trained only on clean datasets like MNIST Dataset

These systems work well for simple inputs but struggle with real handwritten data.

b) Disadvantages of Existing Systems

- Low accuracy for complex handwriting
- Cannot handle variations in:

Writing style

Orientation

Stroke thickness

- Sensitive to noise and background
- Limited generalization to real-world images
- Poor performance on alphabets compared to digits

c) Proposed System

The proposed system uses a **deep learning-based approach** for accurate recognition.

- Uses TensorFlow CNN model
- Dataset: EMNIST Letters Dataset
- Integrates OpenCV for image preprocessing

Key Features:

- Real-time handwritten alphabet recognition
- Robust preprocessing pipeline
- Data augmentation for better generalization
- Confidence-based prediction output

d) Advantages of Proposed System

- High accuracy (~90–96%)
- Handles real-world handwritten inputs
- Robust to noise and distortions
- Fast and efficient prediction
- Supports all 26 alphabets (A–Z)
- Provides top predictions with confidence

PROPOSED SYSTEM ARCHITECTURE

The proposed handwritten alphabet recognition system follows a structured pipeline that integrates image processing and deep learning techniques using TensorFlow and OpenCV. Initially, the system accepts a handwritten character image as input through a digital interface or uploaded file. The input image undergoes preprocessing, which includes grayscale conversion, background normalization, thresholding, contour detection, and cropping to isolate the character. The processed image is then resized to a standard dimension of 28×28 pixels and normalized to improve model performance. To enhance robustness, data augmentation techniques such as slight rotation, cropping, and brightness adjustment are applied. The refined image is then passed to a Convolutional Neural Network (CNN), where multiple convolutional and pooling layers extract important features such as edges, curves, and stroke patterns. Batch normalization and dropout layers are used to improve training stability and prevent over fitting. Finally, fully connected layers with a softmax activation function classify the input into one of the 26 alphabet classes (A–Z). The system outputs the predicted character along with a confidence score and top probable predictions, enabling accurate and real-time handwritten character recognition.

IMPLEMENTATION

Training code-

```
import tensorflow as tf
import tensorflow_datasets as tfds

(ds_train, ds_test), ds_info = tfds.load(
    "emnist/letters",
    split=["train", "test"],
    as_supervised=True,
    with_info=True
)

def preprocess(image, label):
    image = tf.cast(image, tf.float32) / 255.0
    image = tf.reshape(image, (28, 28, 1))
    label = label - 1
    return image, label

def augment(image, label):
    image = tf.image.rot90(image, tf.random.uniform([], 0, 4, dtype=tf.int32))
    image = tf.image.random_crop(image, size=(24, 24, 1))
    image = tf.image.resize(image, (28, 28))
    image = image + tf.random.uniform([], -0.1, 0.1)
    image = tf.clip_by_value(image, 0.0, 1.0)
    return image, label

ds_train = ds_train.map(preprocess).cache()
ds_train = ds_train.map(augment).shuffle(10000).batch(64).prefetch(tf.data.AUTOTUNE)
ds_test = ds_test.map(preprocess).batch(64).cache()
model = tf.keras.Sequential([
    tf.keras.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2D(32, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Dropout(0.25),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Dropout(0.25),
```

```

tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(26, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

callbacks = [
    tf.keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=2)
]

model.fit(ds_train, epochs=5, validation_data=ds_test, callbacks=callbacks)
model.save("emnist_cnn_fixed.keras")

```

Prediction/Testing code-

```

import tensorflow as tf
import numpy as np
import cv2
import matplotlib.pyplot as plt

model = tf.keras.models.load_model(r"C:\Users\rashm\emnist_cnn_model.keras")

img = cv2.imread(r"C:\Users\rashm\AppData\Local\Programs\Microsoft VS
Code\emnist_images\D_55.png", cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Error: Image not found")
    exit()

if np.mean(img) > 127:
    img = 255 - img

_, img = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

```

```

if len(contours) == 0:
    print("No letter detected")
    exit()

cnt = max(contours, key=cv2.contourArea)
x, y, w, h = cv2.boundingRect(cnt)
img = img[y:y+h, x:x+w]

padding = 4
img = cv2.copyMakeBorder(img, padding, padding, padding, padding,
                          cv2.BORDER_CONSTANT, value=0)
img = cv2.resize(img, (28, 28), interpolation=cv2.INTER_AREA)
img = img.astype(np.float32) / 255.0
img = img.reshape(1, 28, 28, 1)

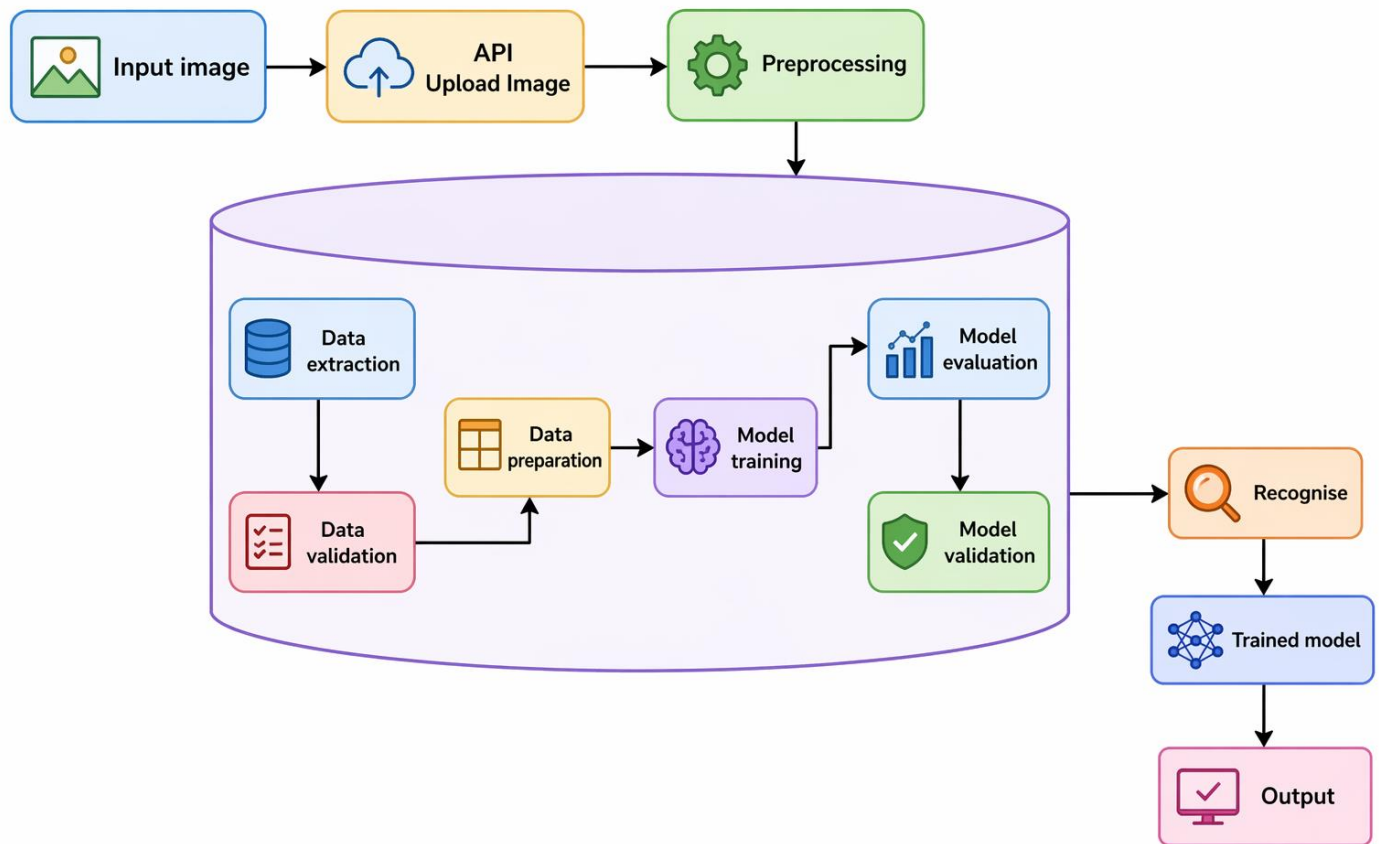
prediction = model.predict(img, verbose=0)
pred = np.argmax(prediction[0])
confidence = prediction[0][pred]
letter = chr(pred + ord('A'))
top3_idx = np.argsort(prediction[0])[-3:][::-1]
top3 = [(chr(i + ord('A')), prediction[0][i]) for i in top3_idx]

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img.reshape(28,28), cmap='gray')
plt.title("Processed Input")
plt.axis('off')
plt.subplot(1, 2, 2)
letters = [chr(i + ord('A')) for i in top3_idx]
probs = [prediction[0][i] for i in top3_idx]
colors = ['green' if l == letter else 'gray' for l in letters]
plt.bar(letters, probs, color=colors)
plt.title(f"Top 3 Predictions\nPrediction: {letter} \n Confidence: {confidence:.2%}")
plt.ylabel("Confidence")
plt.ylim(0, 1)
plt.tight_layout()
plt.show()

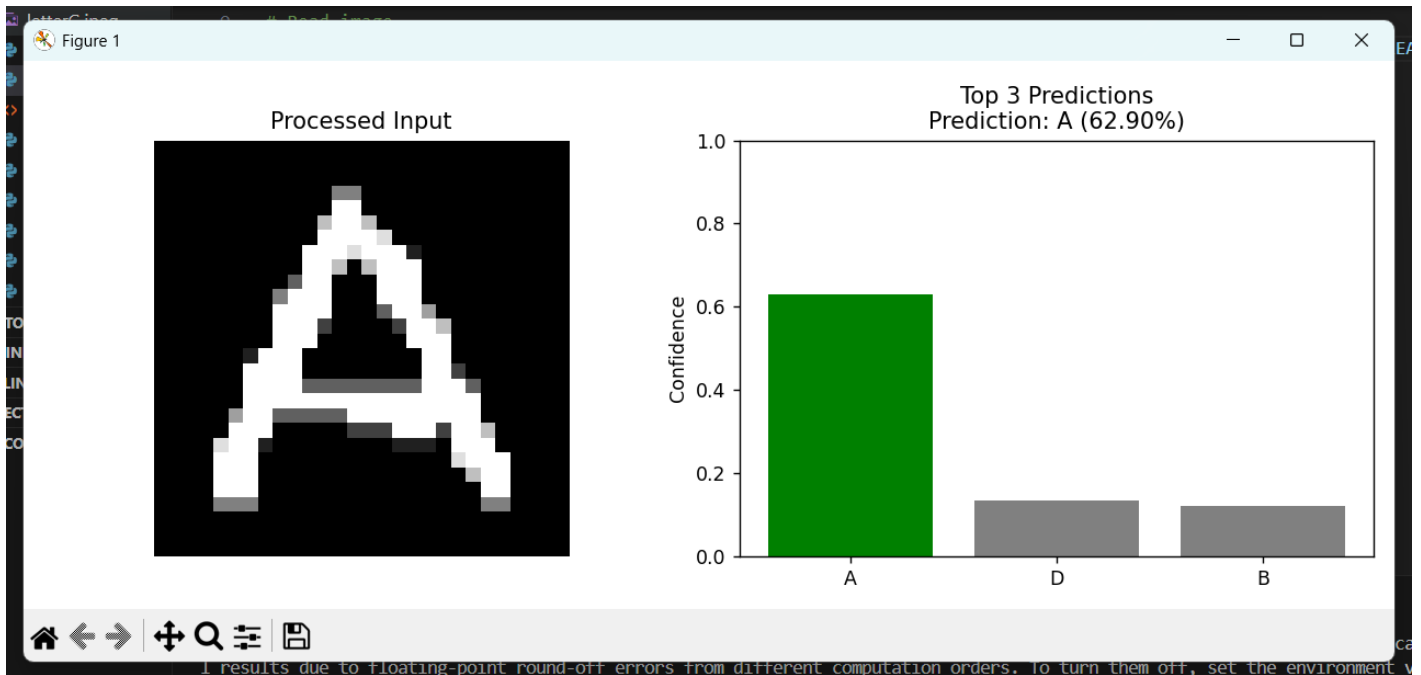
print(f"Prediction: {letter}")
print(f"Confidence: {confidence:.3f}")
print("\nTop 3 predictions:")
for l, prob in top3:
    print(f" {l}: {prob:.3f}")

```

SYSTEM ARCHITECHTURE

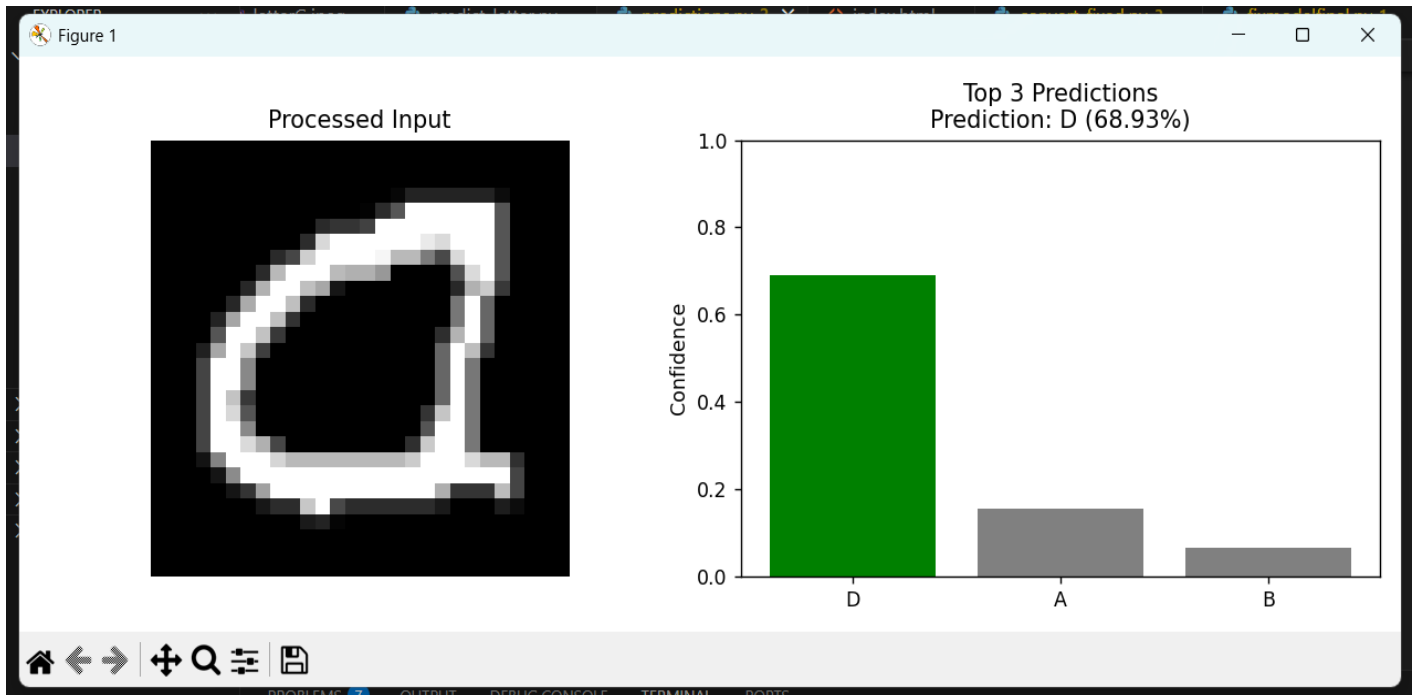


RESULT ANALYSIS



Prediction: A
Confidence: 0.629

Top 3 predictions:
A: 0.629
D: 0.133
B: 0.121



Prediction: D
Confidence: 0.689

Top 3 predictions:
D: 0.689
A: 0.155
B: 0.065

CONCLUSION

The Handwritten Alphabet Recognition system demonstrates how Machine Learning and Deep Learning techniques can be effectively applied to solve real-world problems involving pattern recognition. By using image processing and neural networks such as Convolutional Neural Networks (CNNs), the system can accurately identify handwritten characters from input images.

This project highlights:

- The importance of preprocessing techniques like normalization and noise reduction
- The effectiveness of CNNs in extracting spatial features
- The role of large datasets in improving model accuracy

Despite achieving good accuracy, challenges such as variations in handwriting styles, noise, and ambiguous characters still exist. Future improvements can include:

- Using advanced architectures like ResNet or Transformer-based models
- Increasing dataset diversity
- Implementing real-time recognition using mobile or web applications

Overall, this project provides a strong foundation for further research in Optical Character Recognition (OCR) and intelligent document processing systems.

REFERENCES

TensorFlow Documentation – <https://www.tensorflow.org>

EMNIST Dataset – <https://www.nist.gov/itl/products-and-services/emnist-dataset>

MNIST Database – <http://yann.lecun.com/exdb/mnist/>

OpenCV Documentation – <https://opencv.org>

Research papers on handwritten character recognition from IEEE Xplore / Google Scholar

Kaggle datasets and notebooks on handwritten digit/alphabet recognition.

Tutorials and implementation guides from platforms like GeeksforGeeks.